

Asset Bundles in VS Code

Section 5 · Implementing CI-CD · Quelle: Kurs 8, Modul 08 (Bonus)

Alle bisherigen Beispiele in diesem Themenordner wurden aus einem Databricks-Notebook heraus per `%sh`-Zelle bedient – praktisch für Schulungszwecke, aber nicht der Arbeitsweise, die die meisten Data-Engineering-Teams im Alltag nutzen. In der Praxis wird der Code eines Bundles lokal in einer richtigen Entwicklungsumgebung geschrieben, versioniert und von dort aus deployt. Die **Databricks-Erweiterung für Visual Studio Code** bringt dafür Syntaxhervorhebung, Schema-Validierung, Autovervollständigung für `databricks.yml` sowie Deploy- und Run-Buttons direkt in den Editor – ohne dass man dafür ständig zwischen Browser-Tabs und Terminal wechseln muss.

1. Authentifizierung: Workspace-URL und Personal Access Token

Bevor VS Code mit einem Databricks-Workspace kommunizieren kann, benötigt die Erweiterung zwei Angaben: die Workspace-URL und einen gültigen Berechtigungsnachweis. Für den Einstieg wird meist ein **Personal Access Token (PAT)** verwendet. Die Workspace-URL lässt sich z. B. direkt aus einer laufenden Spark-Session auslesen:

```
lab_databricks_url = f'{spark.conf.get("spark.databricks.workspaceUrl")}/'  
print(lab_databricks_url)
```

Ein PAT wird in der Databricks-Oberfläche über **Einstellungen** → **Developer** → **Access tokens** → **Manage** → **Generate new token** erzeugt. Wichtig dabei: Der Token wird nur einmal im Klartext angezeigt – wird er nicht sofort kopiert und sicher hinterlegt (z. B. in einem Passwort-Manager, keinesfalls im Klartext ins Repository committen), muss ein neuer erzeugt werden. Für den produktiven Einsatz gilt: PATs sollten möglichst eng auf die benötigten API-Bereiche eingeschränkt und regelmäßig rotiert werden; das Scope „alle APIs“ ist ausschließlich für Trainings- bzw. Testzwecke gedacht.

Sobald Workspace-URL und PAT vorliegen, öffnet man in VS Code die Befehlspalette und authentifiziert die Erweiterung gegen den Workspace. Alternativ – und für Teams empfehlenswert – lässt sich die CLI auch mit einem bestehenden **Konfigurationsprofil** (`~/databrickscfg`) oder mit OAuth-basiertem Login verbinden, sodass kein Token im Klartext gepflegt werden muss.

2. Bundles direkt im Editor bearbeiten

Sobald ein Ordner mit einer `databricks.yml` in VS Code geöffnet ist, erkennt die Erweiterung das Projekt automatisch als Bundle. Beim Bearbeiten der YAML-Dateien bietet sie:

- **Schema-Validierung in Echtzeit** – ungültige Schlüssel oder falsch platzierte Zuordnungen werden direkt im Editor als Fehler markiert, ohne dass man erst `databricks bundle validate` über die CLI ausführen muss.
- **Autovervollständigung** für gültige Konfigurationsschlüssel (`bundle` , `resources` , `targets` , `variables` ...) sowie für Referenzen auf andere Ressourcen im selben Bundle.
- **Autovervollständigung für Databricks-Globals** in Python-Notebooks/-Skripten (z. B. `spark` , `dbutils`): Über die Befehlspalette lässt sich mit „Databricks: Configure autocomplete for Databricks globals“ automatisch PySpark installieren und eine passende `__builtins__.pyi` -Datei fürs Projekt anlegen.

3. Der Bundle Resource Explorer: Deploy und Run per Klick

Zentrales Werkzeug der Erweiterung ist der **Bundle Resource Explorer**, eine eigene Seitenleiste in VS Code. Er liest die in `databricks.yml` (und den über `include` eingebundenen Dateien) definierten Ressourcen aus und stellt sie als Baumstruktur dar – Jobs, Pipelines und weitere Objekte lassen sich darüber:

- mit einem Klick deployen, ohne den Terminalbefehl manuell zu tippen,
- direkt aus dem Editor heraus starten und deren Run-Status verfolgen,
- bei Pipelines partiell validieren und aktualisieren, inklusive Einsicht in Pipeline-Ereignisse und Diagnosen,
- mit einem Klick im tatsächlichen, remote Databricks-Workspace öffnen, um dort z. B. den Job-Verlauf einzusehen.

Alle CLI-Befehle, die in den vorherigen Kapiteln über `%sh` -Zellen ausgeführt wurden, funktionieren selbstverständlich unverändert auch im integrierten Terminal von VS Code – der Bundle Resource Explorer ist lediglich eine grafische Abkürzung für dieselben Operationen:

```
# Diese Befehle funktionieren identisch im VS-Code-Terminal wie zuvor im Notebook
databricks bundle validate -t development
databricks bundle deploy -t development
databricks bundle run -t development demo03_job
databricks bundle destroy --auto-approve
```

4. Warum lokal statt im Notebook entwickeln?

Der Wechsel von notebookbasierter Bundle-Arbeit zu einem lokalen Editor bringt vor allem für größere Projekte handfeste Vorteile:

Aspekt	Im Databricks-Notebook (%sh)	In VS Code mit Databricks-Erweiterung
YAML-Fehler erkennen	erst beim Ausführen von <code>bundle validate</code>	sofort während des Tippens (Schema-Validierung)
Versionskontrolle	Umweg über Databricks Git Folders	natives lokales Git, gewohnter PR-Workflow
Deploy/Run auslösen	CLI-Befehl in Notebookzelle	Klick im Bundle Resource Explorer oder Terminalbefehl
Code-Vervollständigung	Notebook-eigene Autovervollständigung	volle IDE-Funktionalität inkl. Databricks-Globals
Lokale Tests	eingeschränkt möglich	pytest & Debugger direkt im Editor nutzbar

Ergänzt aus der offiziellen Databricks-Dokumentation: Die Databricks-Erweiterung für Visual Studio Code ermöglicht es laut Dokumentation, Declarative Automation Bundles direkt zu definieren, zu deployen und auszuführen, um damit CI/CD-Best-Practices auf Lakeflow Jobs, Lakeflow Spark Declarative Pipelines und MLOps Stacks anzuwenden. Der **Bundle Resource Explorer** nutzt dabei die Ressourcendefinitionen aus der Bundle-Konfiguration, um Ressourcen anzuzeigen, mit einem Klick in den Workspace zu deployen und von dort direkt zu den entsprechenden Objekten im Workspace zu navigieren. Für die Erst-Einrichtung der Autovervollständigung für Databricks-spezifische Python-Objekte (`spark` , `dbutils` ...) steht der Befehlspalette-Eintrag „Databricks: Configure autocomplete for Databricks globals“ zur Verfügung.

Quelle: [Declarative Automation Bundles extension features – Databricks-Dokumentation](#)